

Software Security CS-412: Fuzzing Lab

Fuzzing of Open-Source Image/Graphics Libraries

1 Overview

In this project you will set up and execute a coverage-guided fuzzing campaign using **AFL++** against a real-world open-source library. The project comprises two phases:

1. **White-box (source available):** Treat the target as a FOSS project—compile it with AFL++ instrumentation and sanitizers.
2. **Black-box (binary only):** Treat the *same* target as closed-source—compile it with a standard compiler (no instrumentation) and fuzz the resulting binary using AFL++ in QEMU mode (-Q).

Your deliverable is a written report (PDF) answering the evaluation questions in Section 4, accompanied by your complete fuzzing environment (e.g., Dockerfile, Makefile, harness source, patches).

2 Fuzzing Targets

Choose **one** of the following libraries as your fuzzing target:

Library	Lang.	Description
libpng	C	The reference PNG image library.
libsixel	C	SIXEL bitmap graphics encoding and decoding for terminal image display.
libSDL	C	Cross-platform video, audio, graphics development library.

You are free to target any version of the library. Using an older version with known CVEs is acceptable and even encouraged, as it increases the chance of finding real bugs and lets you validate your setup. If you target a recent version and find no crashes, that is also a valid result—provided your methodology is sound.

3 Requirements

Your submission must include the following components.

3.1 Reproducible Fuzzing Environment

- A **Dockerfile** (or equivalent) that sets up the complete fuzzing environment: AFL++ toolchain, target library (compiled with instrumentation and sanitizers), harness binary, seed corpus, and dictionary.
- A **Makefile** with at least **build**, **run/fuzz**, and **clean** targets.

- Your fuzzing harness source code in a `src/` directory.
- Any patches applied to the library (e.g., checksum removal).
- A persistent-mode variant of your harness (e.g., `src/harness_persistent.c`) compiled and used for the performance comparison in Q8.

Reproducibility

Any containerization or automation technology is acceptable (e.g., Docker, Podman, Nix). If you choose not to use containers, you must provide a detailed step-by-step guide that allows a reviewer to reproduce your environment from scratch. **Reproducibility is a graded requirement:** if a reviewer cannot start your campaign, your submission cannot receive full marks.

3.2 Binary-Only (QEMU Mode) Fuzzing Environment

- A second build of the harness and library compiled with a **standard compiler** (`gcc/clang`, no AFL++ instrumentation). Sanitizers must **not** be enabled—this simulates a closed-source binary.
- A Makefile target (`make fuzz-qemu`) that launches AFL++ in QEMU mode (`afl-fuzz -Q`) against this uninstrumented binary.
- A separate findings directory (e.g., `findings-qemu/`) with its own `plot_data` and `afl-plot` output.

3.3 Written Report

A report in PDF format—maximum 4 pages formatted according to the [USENIX style class](#)—answering the evaluation questions in Section 4. Include screenshots of the AFL++ status screen and `afl-plot` output graphs for **both** campaigns. Screenshots must be placed in an **appendix**, which does not count toward the page limit. See Section 5 for more details.

3.4 Campaign Artifacts

The `plot_data` file and `afl-plot` output from both campaigns, each run for at least **30 minutes**.

4 Evaluation Questions

Your report must answer each of the following questions. Each question is graded independently on a **0–5 point** scale. Include evidence from your own campaign (screenshots, metrics, commands, output) where requested.

Rubric descriptors

The point descriptors provided with each question are **indicative**, not exhaustive. They illustrate the level of depth expected at representative score levels (5, 3, 1) but do not capture every possible answer. Reviewers may award intermediate scores (e.g., 2 or 4) and will use their judgment to evaluate answers that fall between the described levels.

4.1 Q1 — Harness Design

(0–5 points)

Identify which library function(s) you chose as the entry point for your harness and justify *why* that function is the most interesting fuzzing target. What other entry points did you consider

(e.g., encoder vs. decoder, high-level convenience API vs. low-level calls)? Why did you reject them—or why would they also be worth fuzzing?

Then walk through your harness code and explain the data flow from the AFL++ input file to the library API call. For every guard in the code (error handling, dimension limits, etc.), explain why it is needed *from a fuzzing perspective*.

Pts	Descriptor
5	Entry point choice justified with API exploration; alternatives discussed; complete data-flow trace; every guard justified with fuzzing rationale.
3	Reasonable entry point chosen but alternatives not discussed, or some guards unexplained.
1	Harness code present but no justification of function choice and no meaningful explanation.

4.2 Q2 — Instrumentation and Sanitizers

(0–5 points)

List every compiler flag and patch you applied to the target library. For each one, explain what it does and what would happen if you omitted it. If you patched the library (e.g., removing checksums), explain the effect on path discovery.

Pts	Descriptor
5	All flags/patches listed with correct, specific explanations of their purpose and impact.
3	Flags listed and broadly correct but missing concrete impact (e.g., “ASan finds memory bugs” without explaining how).
1	Flags listed with no or incorrect explanations.

4.3 Q3 — Seed Corpus and Dictionary

(0–5 points)

Describe your seeds and dictionary. Using the `dictionary` and `havoc/splice` rows from your AFL++ status screen, quantify how many new paths each strategy contributed. Explain what the dictionary entries represent in formal terms (*hint*: think grammar/language theory).

Pts	Descriptor
5	Seeds and dictionary described; numbers from status screen cited and interpreted; formal terms correct.
3	Described but no quantitative analysis from own campaign.
1	Mentioned but not explained.

4.4 Q4 — Campaign Analysis

(0–5 points)

Include your `afl-plot` edges graph and AFL++ status screen screenshot. Report `stability`, `corpus count`, `map density`, and `cycles done`. Describe the shape of the edges curve and argue whether the campaign reached saturation.

Pts	Descriptor
5	All metrics reported with own values; edges curve interpreted with specific timestamps; saturation argued with evidence.
3	Metrics reported but interpretation is generic (not tied to own data).
1	Screenshots included but no analysis.

4.5 Q5 — Crash Triage

(0–5 points)

If crashes were found: Pick one crash and show the full triage — reproduce it, minimize it with `afl-tmin`, obtain an ASan stack trace. Identify the bug type and, if applicable, the corresponding CVE.

If no crashes were found: Prove your setup works by injecting a synthetic bug (e.g., an off-by-one write), re-fuzzing for 60 seconds, and showing AFL++ catches it. Then argue why no real bugs were found.

Pts	Descriptor
5	Complete triage workflow shown with commands and output (or synthetic bug demonstrated + reasoned argument).
3	Partial workflow (e.g., crash reproduced but not minimized) or synthetic bug shown without argument.
1	Crashes mentioned but no triage performed.

4.6 Q6 — Attack Surface Analysis

(0–5 points)

Name two real-world applications that use your target library and describe a concrete attack scenario. Then identify at least two code paths in those applications that your harness does *not* exercise, and explain why these gaps matter for security.

Pts	Descriptor
5	Real applications named; realistic attack scenario described; missed code paths identified with function/file references.
3	Applications named but attack scenario is vague or missed code paths are generic.
1	Surface-level answer without specifics.

4.7 Q7 — Binary-Only Fuzzing with QEMU Mode

(0–5 points)

Run the same fuzzing campaign against an uninstrumented binary using AFL++ QEMU mode (`-Q`). Include the `afl-plot` edges graph and status screen for this campaign. Compare the two campaigns (instrumented vs. QEMU) on three axes: **exec speed**, **edges discovered**, and **corpus count** after the same wall-clock time. Explain *why* the numbers differ by describing how QEMU mode collects coverage compared to compile-time instrumentation.

Pts	Descriptor
5	QEMU campaign runs; side-by-side comparison with concrete numbers on all three axes; correct explanation of how QEMU-based coverage collection works and why it is slower.
3	QEMU campaign runs and numbers reported, but comparison is superficial or mechanism explanation is missing/incorrect.
1	QEMU campaign attempted but no comparison or analysis.

4.8 Q8 — Instrumentation Depth and Performance

(0–5 points)

Report the number of instrumented edges reported by `afl-fuzz` at startup for (a) the library alone and (b) the final harness binary. Explain why these numbers differ. Then compare your campaign’s `map density` to the total instrumented edges and explain why not all edges were reached.

Additionally, measure `exec speed` under three configurations using the same harness: (1) no sanitizer + fork mode, (2) ASan + fork mode, (3) ASan + persistent mode. Report the three numbers and explain the source of each speedup or slowdown.

Pts	Descriptor
5	All edge counts and exec speeds reported; differences correctly explained with reference to instrumentation scope, sanitizer overhead, and fork elimination.
3	Numbers reported but explanations are vague or one configuration is missing.
1	Partial numbers with no meaningful analysis.

5 Submission

Submit a **tarball or zip archive** containing all project artifacts, along with a **link to your repository**, by following the instructions on the Moodle platform.

Repository. Your repository history is part of the evaluation: intermediate commits allow reviewers to assess your working process, so commit early and often. A repository with a single final commit will be penalized.

Report. Your report must use the [USENIX style class](#) and must not exceed **4 pages** (excluding appendices). Figures, code listings, AFL++ screenshots, and `afl-plot` graphs must be placed in an **appendix**, which does not count toward the page limit. The body of the report should reference appendix items where needed (e.g., “see Figure A.1”) but must be self-contained in its argumentation.

Archive contents. Your archive must contain all artifacts referenced in the report (harness source, build files, campaign data, plots, patches). The exact directory layout may vary depending on your tooling, but as a reference, a typical structure looks like:

```
<group-name>-cs412-fuzzing-lab/
  Dockerfile
  Makefile
  report.tex
  report.pdf
```

```
src/  
  harness.c  
patches/                (if any, e.g. nocrc.patch)  
findings/                (instrumented campaign)  
  default/  
    plot_data  
findings-qemu/          (QEMU-mode campaign)  
  default/  
    plot_data  
plot_output/            (afl-plot, instrumented)  
  index.html  
  edges.png  
  exec_speed.png  
  high_freq.png  
  low_freq.png  
plot_output_qemu/       (afl-plot, QEMU)  
  index.html  
  edges.png  
  exec_speed.png  
  high_freq.png  
  low_freq.png
```

6 Tips

- Start early. Building the fuzzing environment and getting `afl-fuzz` to run cleanly takes longer than expected—leave yourself time to actually *analyze* the campaign.
- Commit after every milestone (environment builds, first successful fuzz run, QEMU mode working, persistent mode variant). This protects your work and helps your grade.
- If your campaign finds no crashes, that is a valid result. Read Q5 carefully—you are expected to *prove* your setup is correct, not to fabricate findings.
- The AFL++ documentation is thorough: <https://aflplus.plus/docs/>. The [Fuzzing with AFL++](#) guide is particularly relevant.

CS-412 Software Security — EPFL, Spring 2026. Last updated: April 14, 2026.

Software Security CS-412: Fuzzing libpng

A guided introduction to fuzzing the PNG reference library with AFL++

1 Fuzzing libpng

PNG (Portable Network Graphics) is everywhere. Every web browser, image viewer, game engine, and terminal emulator that displays images almost certainly depends on `libpng`—the official C reference implementation of the PNG standard. It has been around since 1995 and has had plenty of CVEs over the years, several of which were originally found by fuzzing.

This makes it a compelling target: the library is widely deployed, parses a complex binary format, and has a well-documented history of memory-safety bugs. In this document we walk through the PNG format just enough to understand what we are feeding to the fuzzer, and then build up a fuzzing campaign step by step.

2 The PNG Format

A PNG file is not a flat grid of pixels. It is a structured binary container made up of *chunks*, each with a specific role. Understanding this structure is essential for fuzzing, because it tells you where the parser branches, where checksums block mutation, and where the interesting attack surface lives.

2.1 Signature

Every PNG file starts with the same 8-byte signature:

```
89 50 4E 47 0D 0A 1A 0A
```

In ASCII, the bytes `50 4E 47` spell out `PNG`. The remaining bytes are carefully chosen to detect common file-transfer corruptions (e.g., newline conversion between Unix and Windows). If `libpng` does not see this exact signature, it aborts parsing very early—before any chunk processing begins.

From a fuzzing perspective, this means your seed files *must* start with these 8 bytes, or every mutation will be dead on arrival at the very first check.

2.2 Chunks

After the signature, a PNG file is a sequence of **chunks**. Every chunk has the same layout:

Length	Type	Data	CRC
4 bytes	4 bytes	<i>Length</i> bytes	4 bytes

- **Length:** A 4-byte big-endian unsigned integer giving the number of bytes in the **Data** field (not counting Length, Type, or CRC).

- **Type:** Four ASCII characters identifying the chunk. The case of each letter encodes a different property via bit 5 of the ASCII value: the first letter distinguishes *critical* (uppercase) from *ancillary* (lowercase); the second distinguishes *public* from *private*; the third is reserved and must be uppercase; the fourth indicates whether the chunk is *safe to copy* by an editor that does not recognize it.
- **Data:** The chunk payload, interpreted according to the type.
- **CRC:** A CRC-32 checksum computed over the Type and Data fields. This is important for fuzzing—we will come back to it.

2.3 Critical Chunks

A minimal valid PNG contains at least three critical chunks (IHDR, IDAT, IEND). A fourth critical chunk, PLTE, is required for color type 3 (indexed color) and optional for color types 2 and 6:

2.3.1 IHDR — Image Header

Always the first chunk after the signature. It is exactly 13 bytes and contains:

Field	Size	Meaning
Width	4 bytes	Image width in pixels
Height	4 bytes	Image height in pixels
Bit depth	1 byte	Bits per sample (1, 2, 4, 8, or 16)
Color type	1 byte	0=gray, 2=RGB, 3=palette, 4=gray+alpha, 6=RGBA
Compression	1 byte	Always 0 (zlib deflate)
Filter method	1 byte	Always 0 (adaptive filtering)
Interlace	1 byte	0=none, 1=Adam7 interlacing

The combination of color type and bit depth determines how pixel data is stored. `libpng` validates these fields and branches extensively based on them—this is rich attack surface.

2.3.2 IDAT — Image Data

Contains the actual compressed pixel data. The data is a zlib-compressed stream of *filtered* scanlines. Each scanline is preceded by a one-byte filter type (None, Sub, Up, Average, or Paeth) that describes how to reconstruct the raw pixel values. There can be multiple IDAT chunks; their data fields are concatenated before decompression.

The decompression and defiltering pipeline is where many historical CVEs have lived—buffer overflows when palette expansion produces more data than expected, integer overflows in row-size calculations, off-by-one errors in filter reconstruction.

2.3.3 IEND — Image End

A zero-length chunk that marks the end of the file. Simple, but `libpng` still validates its presence.

2.4 Ancillary Chunks

Between IHDR and IDAT (and sometimes after IDAT), a PNG can contain optional ancillary chunks:

Type	Purpose
tRNS	Transparency information
gAMA	Gamma correction value
cHRM	Chromaticity coordinates
sRGB	Standard RGB color space indicator
tEXt	Textual metadata (key-value pairs)
zTXt	Compressed textual metadata
iTXt	International textual metadata (UTF-8)
bKGD	Default background color
pHYs	Physical pixel dimensions
tIME	Last modification time

Each of these has its own parsing code in `libpng`. The critical PLTE chunk is especially interesting from a security perspective: CVE-2015-8126 (fixed in libpng 1.2.54) was a buffer overflow in `png_set_PLTE()` and `png_get_PLTE()` triggered when a crafted image with a small bit-depth in its IHDR chunk caused out-of-bounds access during palette processing.

2.5 The CRC Problem

Every chunk ends with a CRC-32 checksum. When the fuzzer mutates a chunk's data, the CRC no longer matches. For **critical** chunks (IHDR, PLTE, IDAT, IEND), the default behavior is to call `png_error()`, which aborts parsing via `longjmp`. For **ancillary** chunks, the default is to warn and discard the chunk data. In either case, the fuzzer never reaches the deeper parsing code that processes the mutated data.

CRC and fuzzing

Without neutralizing the CRC check, the fuzzer will spend almost all of its time on inputs that either abort (critical chunks) or silently discard mutated data (ancillary chunks). Path discovery will plateau quickly, because mutations to chunk contents rarely lead to new coverage. Patching out the CRC check is strongly recommended for effective fuzzing of `libpng`.

AFL++ ships with a CRC-removal patch in `utils/libpng_no_checksum/`, written for the 1.2.x series. For newer versions, you will need to adapt it—the key function to neutralize is `png_crc_finish()` in `pngutil.c`.

3 Exploring the libpng API

Before writing a harness, you need to understand what `libpng` exposes. Clone the repository and look around:

```
git clone https://github.com/pnggroup/libpng
cd libpng
ls *.h *.c
```

The public API is declared in `png.h`. The key abstractions are:

- `png_structp` — a pointer to the main library state object. Created with `png_create_read_struct()` (for decoding) or `png_create_write_struct()` (for encoding).
- `png_info` — a pointer to image metadata (dimensions, color type, etc.). Created with `png_create_info_struct()`.

Pointer typedefs in newer libpng

In libpng 1.2.x (our target), `png_structp` and `png_infop` are the standard way to declare pointers to the library objects. If you look at the latest development branch (1.8.x), you will see these typedefs marked as [Deprecated] because the structs are becoming fully opaque. The API *functions* (`png_create_read_struct`, `png_read_info`, etc.) remain unchanged, so the call sequence below works across all versions—only the type names for local variables may change in future releases.

The typical decoding call sequence is:

```
png_structp png = png_create_read_struct(...);
png_infop   info = png_create_info_struct(png);

/* set up input source */
png_set_read_fn(png, my_data, my_read_callback);

/* read header and metadata chunks */
png_read_info(png, info);

/* optionally apply transformations */
png_set_expand(png);      /* palette -> RGB */
png_set_strip_16(png);    /* 16-bit -> 8-bit */
png_set_gray_to_rgb(png); /* grayscale -> RGB */
png_read_update_info(png, info);

/* read pixel data */
png_read_image(png, row_pointers);

/* read post-IDAT chunks */
png_read_end(png, NULL);

/* clean up */
png_destroy_read_struct(&png, &info, NULL);
```

But decoding is not the only option. libpng also exposes:

- **Encoding functions:** `png_write_image()`, `png_write_info()`, etc. These take structured data and produce PNG bytes. Fuzzing the encoder means feeding it random dimensions, color types, and pixel buffers.
- **Low-level chunk access:** internal functions such as `png_read_chunk_header()` and `png_push_read_chunk()` (progressive/incremental reading). The public API exposes chunk-level hooks via `png_set_read_user_chunk_fn()`.
- **Simplified API** (libpng 1.6.0+): `png_image_begin_read_from_memory()`, `png_image_finish_read()`. A higher-level convenience interface that wraps the full pipeline.

When choosing your harness entry point, consider: which function processes the most untrusted data with the most complex parsing logic? Which functions are actually called by real applications? Where have historical bugs been found?

4 Building the Fuzzing Environment

4.1 Downloading and Patching libpng

Pick a version. For this walkthrough we use libpng 1.2.56, which is compatible with the AFL++ CRC patch. Note that the specific CVE-2015-8126 PLTE overflow was fixed in 1.2.54, but 1.2.56 still contains other vulnerabilities (e.g., CVE-2016-10087, a NULL-pointer dereference in `png_set_text_2()`):

```
wget https://download.sourceforge.net/libpng/libpng-1.2.56.tar.gz
tar xf libpng-1.2.56.tar.gz
cd libpng-1.2.56
```

Apply the CRC patch:

```
patch -p0 < /path/to/AFLplusplus/utils/libpng_no_checksum/libpng-nocrc.patch
```

If you use a newer libpng version (1.6.x), this patch will not apply cleanly. The minimal manual fix is to make `png_crc_finish()` return 0 immediately. In `pngutil.c`, find the function and insert `return 0;` as the first line of the body.

4.2 Compiling with Instrumentation

Build libpng as a static library with AFL++ instrumentation and AddressSanitizer:

```
CC=afl-clang-fast \
CXX=afl-clang-fast++ \
CFLAGS="-fsanitize=address -g -O1" \
LDFLAGS="-fsanitize=address" \
./configure --disable-shared --prefix=$(pwd)/install
make -j$(nproc)
make install
```

When you later run the instrumented binary (or compile with `AFL_DEBUG=1`), AFL++ reports the total number of instrumented edges. The count across all libpng source files tells you how many edges the fuzzer *could* reach. Keep this number—you will need it later to reason about your campaign’s map density.

Static linking

The `--disable-shared` flag is strongly recommended. Although AFL++ *can* instrument shared libraries (e.g., via `AFL_PRELOAD`), static linking avoids pitfalls such as `LD_LIBRARY_PATH` management, accidental system-wide installation, and `dlopen()`-related timing issues with the fork server.

4.3 Writing a Harness

A fuzzing harness is a small `main()` function that bridges AFL++ and the library. Its job is simple: read the fuzzer’s input, pass it to the library, handle errors gracefully, and exit.

Some things to think about when writing yours:

- **Input delivery:** AFL++ can deliver input as a file on disk (@@) or via stdin. File-based is simpler to start with.
- **Error handling:** libpng uses `setjmp/longjmp` for error handling. If you don’t set up the `setjmp` handler, every malformed input will cause an unhandled `longjmp`, and AFL++ will log it as a crash. You will drown in false positives.

- **Resource limits:** A mutated PNG header might claim dimensions of 65535×65535 . Think about what happens when `libpng` tries to allocate memory for that.
- **Transformation depth:** The more `png_set_*` transformations you enable, the more code paths the fuzzer can reach. Historical bugs like CVE-2015-8126 live in the palette expansion pipeline, which is only exercised if you call `png_set_expand()`.

Compile the harness against your instrumented static library:

```
afl-clang-fast harness.c \  
-I./install/include \  
-L./install/lib \  
-lpng12 -lz -lm \  
-fsanitize=address -g -O1 \  
-o png_fuzz
```

Note the total number of instrumented edges reported when the harness binary starts (visible in the AFL++ status screen or with `AFL_DEBUG=1`). Compare it to the library-only count—the difference tells you how many edges the harness itself contributes.

Sanity-check the binary before fuzzing:

```
./png_fuzz some_valid.png  
echo $? # should be 0
```

If `libpng` prints an error message but the process exits cleanly (exit code 0), your error handling is working correctly. If you see a segfault or ASan report on a valid PNG, fix your harness before proceeding.

5 Seeds and Dictionary

5.1 Seed Corpus

AFL++ needs at least one non-empty seed file to start. The quality of your seeds directly affects how quickly the fuzzer discovers deep code paths.

AFL++ ships with a small set of PNG test cases in `testcases/images/png/`. These are minimal valid PNGs, so the fuzzer starts with basic coverage but still has to discover many format features through mutation.

A better strategy is to provide a diverse set of small, valid PNGs that exercise different features of the format: one grayscale, one palette, one RGB, one with alpha, one interlaced, one 16-bit depth. Keep them small (under 10 KB)—large seeds slow down calibration and execution without adding proportional coverage.

If you already have a large collection of PNGs, use `afl-cmin` to minimize the corpus before fuzzing:

```
afl-cmin -i large_corpus/ -o minimized/ -- ./png_fuzz @@
```

This keeps only the smallest set of inputs that together achieve the same total coverage.

5.2 Dictionary

AFL++ can use a dictionary of format-specific tokens to guide mutations. The PNG dictionary is at `dictionaries/png.dict` and contains entries like:

```
header_png="\x89PNG\x0d\x0a\x1a\x0a"
section_IHDR="IHDR"
section_PLTE="PLTE"
section_IDAT="IDAT"
section_IEND="IEND"
section_gAMA="gAMA"
section_tEXt="tEXt"
```

These are the chunk type names—the meaningful atomic tokens that the parser dispatches on. Without the dictionary, the fuzzer would have to discover the 4-byte sequence `IHDR` by random bit flips (a 1 in 2^{32} chance per position). With it, AFL++ can insert these tokens directly during mutation, dramatically accelerating the discovery of chunk-handling code paths.

Pass the dictionary with `-x`:

```
afl-fuzz -i seeds -o findings -x png.dict -- ./png_fuzz @@
```

6 Running the Campaign

6.1 Source-Based (Instrumented)

With everything in place, launch the fuzzer:

```
afl-fuzz -i seeds -o findings -x png.dict -- ./png_fuzz @@
```

You should see the AFL++ status screen with `corpus count` climbing within the first minute. Key things to watch:

- **stability:** Should be 100% or very close. If it is low, your harness has non-deterministic behavior.
- **exec speed:** Typical values range from a few hundred to over 1000 exec/sec depending on your hardware. ASan adds overhead; without it, speeds are significantly higher.
- **corpus count:** Should grow rapidly in the first few minutes, then slow down.
- **map density:** Shows what fraction of instrumented edges the fuzzer has reached.

Let it run for at least 30 minutes. Use `afl-plot` to visualize the results:

```
afl-plot findings/default/ plot_output/
```

6.2 Binary-Only (QEMU Mode)

To simulate a closed-source scenario, build a second copy of libpng with a standard compiler—no AFL++ instrumentation, no sanitizers. Keep the CRC patch applied so that the comparison with the instrumented campaign isolates the effect of QEMU overhead:

```
cd libpng-1.2.56
make distclean
CC=gcc CFLAGS="-g -O1" \
  ./configure --disable-shared --prefix=$(pwd)/install_vanilla
make -j$(nproc) && make install
```

Then compile the harness against this vanilla installation:

```
gcc harness.c \
-I./install_vanilla/include \
-L./install_vanilla/lib \
-lpng12 -lz -lm \
-g -O1 \
-o png_fuzz_qemu
```

Then run AFL++ in QEMU mode:

```
afl-fuzz -Q -i seeds -o findings-qemu -x png.dict -- ./png_fuzz_qemu @@
```

The `-Q` flag tells AFL++ to run the target under QEMU user-mode emulation, which translates basic blocks at runtime and injects coverage tracking. This is slower than compile-time instrumentation because every basic block must be translated on first encounter and the emulation layer adds overhead to every instruction. Expect exec speed to drop by roughly $2\text{--}5\times$ compared to the instrumented build.

QEMU mode and sanitizers

By default, QEMU mode does not include AddressSanitizer—since you are fuzzing an uninstrumented binary, there is no compiler-inserted sanitizer to catch subtle memory errors. The fuzzer can only detect crashes that result in signals (SIGSEGV, SIGABRT, etc.). Bugs that corrupt memory silently will go unnoticed.

Modern AFL++ can partially bridge this gap via **QASan** (`AFL_USE_QASAN=1`), which hooks memory allocations at runtime to detect issues like heap overflows and use-after-free even in binary-only targets. Exploring QASan is optional for this lab, but worth knowing about.

Why keep the CRC patch for QEMU mode?

In a real closed-source scenario you would *not* be able to patch out checksums, and the fuzzer would waste most of its time generating inputs that fail CRC validation—never reaching the deeper parsing code. Techniques such as custom mutators or protocol-aware fuzzing can mitigate this, but they are beyond the scope of this lab. We keep the CRC patch here so that the QEMU campaign explores comparable code paths and you can meaningfully compare exec speed and edge discovery between the two modes.

6.3 Persistent Mode

In the default fork server mode, AFL++ forks a new process for every test case. Persistent mode eliminates this overhead by looping inside the same process:

```
__AFL_FUZZ_INIT();

int main(int argc, char **argv) {
    __AFL_INIT(); /* deferred fork server */
    unsigned char *buf = __AFL_FUZZ_TESTCASE_BUF;

    while (__AFL_LOOP(10000)) {
        int len = __AFL_FUZZ_TESTCASE_LEN;
        /* feed buf/len to libpng, clean up state */
    }
    return 0;
}
```

The `__AFL_LOOP(N)` macro runs the body `N` times before the process exits and AFL++ restarts it, giving you the speed of in-process fuzzing with periodic state resets. Typical speedups range from $2\times$ to $20\times$ over fork mode, depending on the target.

The tradeoff is that if the library leaks state between iterations (global variables, cached allocations), the harness may behave differently on iteration 1000 than on iteration 1. This can lower stability. If you observe stability dropping after switching to persistent mode, reduce `N` or add more aggressive cleanup between iterations.

7 Triage

If AFL++ finds crashes, they appear in `findings/default/crashes/`. The standard triage workflow is:

1. **Reproduce:** Run the crashing input through your harness to confirm it still crashes.

```
./png_fuzz findings/default/crashes/id:000000,sig:11,...
```

2. **Minimize:** Use `afl-tmin` to shrink the input to the smallest version that still triggers the same crash. This makes root-cause analysis much easier.

```
afl-tmin -i crash_file -o minimized.png -- ./png_fuzz @@
```

3. **Diagnose:** Run the minimized input with ASan symbolization enabled to get a human-readable stack trace:

```
ASAN_OPTIONS=symbolize=1 ./png_fuzz minimized.png
```

The ASan output will tell you the bug type (heap-buffer-overflow, stack-buffer-overflow, use-after-free, etc.), the exact source location, and the allocation context.

If your campaign finds no crashes after 30 minutes, that does not mean your setup is broken. It may simply mean the library version you chose is well-tested. Refer to the lab evaluation guidelines for how to handle this case.

8 Useful References

- **PNG specification:** <http://www.libpng.org/pub/png/spec/1.2/PNG-Contents.html>
- **libpng manual:** <http://www.libpng.org/pub/png/libpng-manual.txt>
- **libpng source:** <https://github.com/pnggroup/libpng>
- **AFL++ documentation:** <https://aflplus.plus/docs/>
- **AFL++ fuzzing in depth:** https://aflplus.plus/docs/fuzzing_in_depth/
- **AFL++ libpng CRC patch:** `AFLplusplus/utils/libpng_no_checksum/`
- **AFL++ PNG dictionary:** `AFLplusplus/dictionaries/png.dict`
- **CVE-2015-8126** (palette overflow in libpng): <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2015-8126>

Here, #0 and #1 are just identifiers for colors. The 2 tells us that we are using RGB¹, and the values that follow are intensities between 0 and 100 (not 0-255!). So in this example, #0 is red and #1 is white.

By now, you can probably tell by now where this is going: We are going to make the EPFL logo in sixel by hand :)

Now the interesting part: how do we actually encode pixels?

Instead of storing pixels row by row, Sixel works column-wise. Each printable character represents a vertical strip of 6 pixels. To decode it, you take the ASCII value of the character and subtract 63². You get a 6-bit number, where each bit represents which pixels the “six”el are active. Say we have the sixel 0b110101 (0x35). The least significant bit is at the top while the most significant bit is at the bottom.

0x35	P
1	x
0	o
1	x
0	o
1	x
1	x

The printable character used to represent this column is $0x35 + 0x3F = 0x74$: ASCII `t`.

With this in mind, we can take a look at the fist line which nicely shows the encoding of a sixel-line: `[color][sixel-line-array]`.

```
#0~~~~~$
```

We switch to color #0 (red) and then repeat ~ 36 times. In other words, we draw 36 vertical columns, each with all six pixels set. This gives us a solid red strip. The \$ at the end resets the cursor to column 0, but stays on the same row. This make us draw on top of our red canvas.

The next line switches to white and starts drawing more interesting patterns:

```
#1?}}eeeeee?}}}}eeee}}[?}}}}eeeeee?}}}}?????}-
```

Each of these characters corresponds to a different 6-bit pattern, so instead of a full column, we now get partial columns. These overwrite parts of the red background, effectively “drawing” shapes.

Chars:	?}}}}eeeeee	?}}}}eeee}}[	?}}}}eeeeee	?}}}}?????}
Row 0:				
Row 1:	.xxxxxxx	..xxxxxxx.	..xxxxxxx	..xx.....
Row 2:	.xxxxxxx	..xxxxxxxx	..xxxxxxx	..xx.....
Row 3:	.xx.....	..xx....xx	..xx.....	..xx.....
Row 4:	.xx.....	..xx....xx	..xx.....	..xx.....
Row 5:	.xxxxxxx	..xxxxxxx.	..xxxxxxx	..xx.....

Here, x means that the pixel is set in the current color (white), while . means that the pixel is not set. Since this line is drawn after the red background, the x positions overwrite the red pixels with white ones.

¹1 is HLS, where colors are given as hue, lightness, and saturation

²Check `man ascii` for reference.

At the end of the line, we see a `-`. Unlike `$`, which keeps us on the same row, `-` moves the cursor down by one sixel row³. This is how we start drawing the next part of the image.

We repeat the pattern: we draw another red background row, and then overlay it with white pixels. By stacking these 6-pixel-high bands, we gradually build up the full image.

Finally, the image is terminated with:

```
ESC \
```

At this point, the terminal has everything it needs to reconstruct the image.

To get an intuition for what this actually looks like, it helps to render the Sixel into a png image. The final image generated from the tool `sixel2png`⁴ is the following:



1.2 Run-Length encoding

```
ESC Pq
#0;2;100;000;0
#1;2;100;100;100
#0!36~$
#1?}}eeeeee?}}eeeeee}[[?}}eeeeee?}}?????-
#0!36~$
#1?^XXXXX??^@@@??^@@@??^WWWW? $
ESC \
```

As you can probably tell this saves some storage when we encode long sequences. However, this also adds complexity to the decoding.

2 Starting the Fuzzing Lab

Well, now that we know what we are dealing with, we actually want to look for some bugs. So lets clone the repo, and try to build it.

```
git clone https://github.com/saitoha/libsixel libsixel
cd libsixel
./configure
make -j
```

Well, what now? I suggest you to look around and find out what the repository looks like. Often this gives you a hint on what functionality makes sense to fuzz.

³You might recognize `$` as carriage return and `-` as line feed.

⁴Which you should take a closer look at!

Finiding Starting Points

We can make some observations just by browsing the repository:

- The converters and the example show how the library is used.
- Furthermore, it makes sense to look at the images/issues directory, since it is a good baseline to see what the authors were working with.
- We also find that this library has a number of bindings to e.g. Perl, PHP, Ruby, Python. This is actually pretty interesting, since we find out that a vulnerability in libsixel can also impact those languages.

2.1 Finding the Fuzzing Harness

We have to get the main entry point to the library. But what funcionts could we pick?

Using the “right” API

We take a look at `include/sixel.h` which defines the library’s entrypoints. Obviously, the prime target is what does the decoding, as we assume attacker controlled input is a sixel encoded image that has to be displayed on e.g. a terminal or has to be encoded to a png/jpeg/... For this reason it is safe to assume that an attacker would use this API for exploiting an application.

We see more functions that make (more or less) sense to use as harnesses. Decoding is of course not the only API that makes sense for fuzzing: Consider `sixel_helper_normalize_pixelformat()`: It does a number of conversions with width/height arithmetics and has some assumptions about the destination buffer. Getting non-decoding harnesses right is more difficult since the dimensions are derived from the input and violating those prerequisites might lead to false-positive crashes. Fuzzing/security testing is also about how much effort we would like to invest and a decoder is the most low-hanging fruit.

So let’s take a closer look to decoding for now. We want to pick a good target, but we find multiple functions that have `decoding` in their name.

2.2 Getting to Work

Well, now that we know what we fuzz, we want to do it. Let’s stick for now to a harness that does decoding, since that is the most straightforward thing to do.

We could e.g. use the abovementioned `sixel2png` since it calls `sixel_decoder_decode`. It turns out that this function also calls `sixel_decode_raw`. However, as we will find out later, writing our own harness is a superior design choice.

2.2.1 Custom Harnessing

We can build a minimal harness that just takes input from `stdin` and calls the `sixel_decode_raw` function on it.

To sanity check our function, we check our EPFL test file:

```
cat ../../seeds/epfl.six | ./sixeldecode
Code: 0
Out: 37 12 3
```

To debug our harness, we collected and printed the return code. We get a 0 return code (OK) and 37x12 pixels with 3 colors, which matches our image (+1 color).

Performance of Custom Harnesses

Comparing the performance of a hand-written harness that calls `sixel_decode_raw` and `sixel2png`, we observe a significant performance gain. This gain comes from skipping all the functionality, which is non-decoding related. This way we don't only speed up the execution but also cut out a lot of complexity for fuzzing.

2.3 Invoking the Fuzzer

Our fuzzing harness works with `libsixel`. Now it's time to use `AFL++` to do the work. Having `afl++` installed, we can actually put it to work on the “black-box” binary. However, to do this, we need some seeds first!

Using “good” seeds

Generally, the Fuzzer's performance stands and falls with the quality of the seeds:

- Any small example, like the EPFL one, that is crafted by hand makes sense.
- Some quick searching on wikipedia will also reveal some example Sixel images that (i) can be used and (ii) can be adapted: Sixel Sample images.

Given the seeds, we can *actually* start now:

```
AFL_SKIP_CPUFREQ=1 \
AFL_AUTORESUME=1 \
afl-fuzz -n -i "SEEDS" -o "OUT_DIR" -- "HARNESS"
```

2.4 Fuzzing with Feedback

Now we compile with `AFL`, to get at least rid of one of the above issues. We find multiple modes for instrumenting; two of them are particularly interesting:

- `afl-clang-fast` in **CLASSIC** mode: the “traditional” scheme.
CLASSIC: `map[cur_loc ^ (prev_loc >> 1)]++`
 Although improbable, it is possible to have collisions, especially if the program is big.
- `afl-clang-lto`
LTO: Link Time Optimization
 Link-time edge IDs with non-colliding edge coverage and optional edge-ID documentation.

The tradeoff is that linking is slower with LTO. On the flip-side, we get quite 1:1 edge-bit mappings. LTO is usually the preferred choice, if you bring some compile time with you.

Try to build `libsixel` with `afl-clang`, and compile it statically:

```
afl-clang-fast main_harness.c \
-I../include \
../src/.libs/libsixel.a -lcurl \
-o sixeldecode
```

Note the total number of instrumented edges reported when the harness binary starts (visible with `AFL_DEBUG=1`). Compare it to the library-only count. The difference tells you how many edges the harness itself contributes.

Sanity Checking Edge Counts

At this point it is worth doing a small sanity check before starting the fuzzing campaign with coverage instrumentation. We find that the statically compiled version has less edges than the library itself!

The reason is that we are linking against a **static archive**. A static library is not copied whole into the executable. Instead, the linker only pulls in the object files that are needed to satisfy the symbols referenced by our harness. Since our harness only calls the decoder entry point, only the decoder-related subset of `libsixel` ends up in the executable. Encoder code, unrelated helpers, writers, loaders, and other unused parts of the archive are simply left out.

```
afl-clang-fast -O2 -g -I ../../include \
  main.c \
  ../../src/.libs/libsixel.a -lcurl \
  -o sixeldecode
```

We get all of them by asking our linker to keep the wholesale static archive in the binary:

```
afl-clang-fast -O2 -g -I ../../include \
  main.c \
  -Wl,--whole-archive ../../src/.libs/libsixel.a -Wl,--no-whole-archive \
  -lcurl -lpng -lm -ljpeg \
  -o sixeldecode
```

This yields all edges, however, we find that most of them are not reachable! This is because there is no callsite in the harness that executes any instrumented instructions.

2.5 Optimizing

Well, we figured out one thing at first: restarting the binary all the time is not a good thing. For this reason, we consider persistent mode fuzzing:

Persistent Mode Fuzzing

```
__AFL_INIT();
while (__AFL_LOOP(1000)) {
    unsigned char *buf = __AFL_FUZZ_TESTCASE_BUF;
    int len = __AFL_FUZZ_TESTCASE_LEN;
    // Your code here!
}
```

This modification should provide us with a considerable improvement in performance compared to the baseline which does not use persistent mode.

However, an interesting question to ask is where the macros come from! They are defined in the runtime which is linked by `afl-clang-fast`. This shows an inherent benefit of using a customized compiler: no linking modification is necessary to get persistent fuzzing to work. In fact, we only need to use already linked symbols and they are picked up by the runtime, yielding a large performance benefit for stateless library fuzzing.

2.6 Going Opaque Again: AFL QEMU Mode

In many practical scenarios, a given program is (unfortunately) not open-source, and therefore, does not allow the use of instrumentation.

The `-Q` flag tells AFL++ to run the target under QEMU user-mode emulation, which translates basic blocks at runtime and injects coverage tracking. This is slower than compile-time instrumen-

tation because every basic block must be translated on first encounter and the emulation layer adds overhead to every instruction. Expect the execution speed to drop significantly compared the compiler-instrumented build.

QEMU mode and sanitizers

By default, QEMU mode does not include AddressSanitizer—since you are fuzzing an uninstrumented binary, there is no compiler-inserted sanitizer to catch subtle memory errors. The fuzzer can only detect crashes that result in signals (SIGSEGV, SIGABRT, etc.). Bugs that corrupt memory silently will go unnoticed.

Modern AFL++ can partially bridge this gap via **QASan** (AFL_USE_QASAN=1), which hooks memory allocations at runtime to detect issues like heap overflows and use-after-free even in binary-only targets. Exploring QASan is optional for this lab, but worth knowing about.

In order to run QEMU AFL mode, compile your harness once again without AFL++ and run it with the following command:

```
afl-fuzz -Q -i IN -o OUT-QEMU -- ./sixeldecode @@
```

2.7 Triage and Finding Bugs

With all the knowledge from this lab, you should be able to find bugs in `libsixel`. Congratulations!

However, you have to give us at least one **proof-of-concept** (POC) that crashes `libsixel`. Don't worry, if your fuzzing campaign does not yield a new bug! In this case browse the repository for some issues and reproduce one :)

Producing PoCs

Consider reproducing some *open* issue in the encoder: E.g. GH issue 220 or GH issue 111. This requires the students to checkout to a pinned commit and run the harness.

Software Security CS-412: Fuzzing libSDL

A guided introduction to fuzzing the SDL library with AFL++

1 Fuzzing libSDL

This is an introduction to SDL, one of the fuzzing targets available for this lab. For more details, refer to the SDL3 wiki (for SDL 3.0 and higher) or the SDL2 wiki, depending on which version you have chosen to fuzz.

As said in the description on their official webpage, it is a cross-platform library designed to provide low level access to audio, keyboard, mouse, joystick, and graphics hardware via OpenGL and Direct3D. It is written in C and is used by video playback software, emulators, and various games, probably the most famous being Noita.

1.1 Introduction

The description above deserves further clarification. As you were probably taught in your operating systems course, all of the mentioned hardware is at a fundamental level handled by the kernel, which receives interrupts and dispatches them to handlers which process input from these devices. Userspace programs access it via the API provided by the operating system. Unfortunately, this means that writing a program that uses, for example, mouse input is very platform-dependent. In other words, by accessing the mouse input through the operating system's API, you will need to write different code for Windows, for Linux and for macOS, which, as you may imagine, is a tedious and error-prone task. To resolve this issue, people have resorted to the standard computer science solution of building abstractions and wrappers. SDL is one of these abstractions; it is a rather large library that someone else wrote that deals with platform-specific things and provides a simpler, unified API to the programmer that is using it.

In addition to allowing you to write cross-platform programs more easily, another benefit of using SDL is that it abstracts away the complexity of graphics APIs (OpenGL and Direct3D). These APIs provide very low-level access to graphics hardware (for example, your NVidia RTX 5090), which makes them powerful, but on the other hand makes them rather complicated to use. To put it simply, displaying an image from a file without SDL (relying on OpenGL for rendering and Windows API for handling window creation) requires hundreds of lines of code, while using SDL it can be done in a bit more than 60 lines.

1.2 Example Usage

Here is an example program¹ that opens a window, loads a bitmap image called lettuce.bmp and displays it:

```
#include <SDL3/SDL.h>
#include <iostream>
int main(int argc, char* argv[]) {
```

¹<https://glusoft.com/sdl3-tutorials/display-image-sdl3>

```

SDL_Init(SDL_INIT_VIDEO);
SDL_Window* win = SDL_CreateWindow("SDL3 Image", 640, 480, 0);
if (win == nullptr) {
    std::cerr << "SDL_CreateWindow Error: " << SDL_GetError() << std::endl;
    SDL_Quit();
    return 1;
}

SDL_Renderer* ren = SDL_CreateRenderer(win, NULL);
if (ren == nullptr) {
    std::cerr << "SDL_CreateRenderer Error: " << SDL_GetError() << std::endl;
    SDL_DestroyWindow(win);
    SDL_Quit();
    return 1;
}

SDL_Surface* bmp = SDL_LoadBMP("lettuce.bmp");
if (bmp == nullptr) {
    std::cerr << "SDL_LoadBMP Error: " << SDL_GetError() << std::endl;
    SDL_DestroyRenderer(ren);
    SDL_DestroyWindow(win);
    SDL_Quit();
    return 1;
}

SDL_Texture* tex = SDL_CreateTextureFromSurface(ren, bmp);
SDL_DestroySurface(bmp);
if (tex == nullptr) {
    std::cerr << "SDL_CreateTextureFromSurface Error: " << SDL_GetError() << std::endl;
    SDL_DestroyRenderer(ren);
    SDL_DestroyWindow(win);
    SDL_Quit();
    return 1;
}

SDL_Event e;
bool quit = false;
while (!quit) {
    while (SDL_PollEvent(&e)) {
        if (e.type == SDL_EVENT_QUIT) {
            quit = true;
        }
    }
    SDL_RenderClear(ren);
    SDL_RenderTexture(ren, tex, NULL, NULL);
    SDL_RenderPresent(ren);
}
SDL_DestroyTexture(tex);
SDL_DestroyRenderer(ren);
SDL_DestroyWindow(win);
SDL_Quit();
return 0;
}

```

It may seem intimidatingly long at first, but if you look more closely, you will see that a big part of the code is just error checking (i.e., checking whether the window/renderer/surface/texture was created successfully).

1.3 Setting Things Up

Now that you have understood what SDL is and what its usage looks like, it is time to set it up on your system. It is recommended to start working in a Docker container immediately because that is what the submission format requires.

You can download any SDL release from the releases page on the GitHub repository. Make sure you download the source code, not the pre-built version. Then, unarchive it and use the terminal to navigate to the extracted folder contents. There, run the following commands:

```
./configure  
make  
make install
```

Note that `make install` may require `sudo` privileges. After that, the library is ready to use. You can link your program to it using the compiler option `-lSDL3` or `-lSDL2`:

```
clang target.cpp -o target -lSDL2
```

As with most C/C++ repositories, you can find the header files in the `include/` folder and source files in the `src/` folder. SDL has many different components (audio, filesystem, GPU handling, input, etc.); you can find an exhaustive list here (for SDL3) or here (for SDL2).

Finally, here is something that may interest you in the context of this lab. Documentation for the SDL function for loading PNG images, `SDL_LoadPNG` contains the following warning: “*This is intended as a convenience function for loading images from trusted sources. If you want to load arbitrary images you should use `libpng` or another image loading library designed with security in mind.*” Who knows, maybe there are other components of SDL that would deserve such a warning?

And with that, you are now all set to complete the lab.